

RTS.AI GitHub Mastery Path — Curriculum & Design Spec v1.1

For: Adam Cornelius, owner/operator, RUN Trading Systems (RTS.AI) LLC **Purpose:** Full curriculum design for a visual, immersive GitHub course teaching everything the terminal can't show: what things *look like*, where they *live*, and why each screen exists. **Companion document:**

PROMPT_GitHub_Mastery_Path_Replit_Build_v1_1_2026_08_14.md — the standalone prompt for a fresh Replit Agent chat to actually build this. **Status:** DRAFT for your review. Nothing here is built yet. This is the blueprint; the Replit prompt is the construction order.

What changed in v1.1: you flagged that gating the visual course behind the CLI dojo would cause exactly the drop-off problem you're trying to avoid — commands are genuinely hard for a newb on day one, and a beginner who stalls on a terminal prompt quits before they ever see what GitHub actually does. So the structure changed: the CLI dojo is no longer a prerequisite "Tier 0." It's now the **Command Test Center** — a standalone, optional, always-reachable practice arena, foldable directly into the same app, that a learner can try at any point and return to as many times as they want, with zero pressure to pass it before moving on. Section 3 and Section 6 below carry the full rewrite; everything else (the six visual tiers, the research basis, the design rules) is unchanged from v1.0.

1. The problem this solves

You finished the CLI dojo — real progress, real commands, real muscle memory. But you said the honest thing: you needed Replit's own chat agent to get through it, and what you were missing wasn't the commands, it was the **picture**. A terminal tells you *what to type*. It never shows you *where you are*, *what it looks like on the other end*, or *why anyone built it that way*. PowerShell (and Bash, and every terminal) strips context out by design — that's what makes it fast for experts and disorienting for everyone else on day one.

This course exists to put the picture back. Once you can see a pull request, a branch protection rule, an Actions run — and understand *why* each exists and *what problem it solves* — the commands stop being incantations and start being obvious. "Oh, git push just sends this to the same GitHub repo I'm looking at right now" lands completely differently once you've actually looked at it.

2. Design principles (non-negotiable, carried into every module)

1. **Context before command.** Every screen is introduced with what it's *for* and *who uses it* and *why*, before any button gets clicked.
2. **Your own use case, always in frame.** Examples throughout use "your company's repos" — private code, RTS.AI documents, a future contractor's pull request — not generic open-source strangers. You should never have to translate a tutorial's toy example into your own situation; the toy example *is* your situation.
3. **Nothing here is GitHub's trademarked interface.** The simulated screens are built to teach the same information architecture and interaction patterns real GitHub uses — the tabs,

the diff view, the review panel, the Actions log — but with RTS’s own original visual design, not GitHub’s logo, wordmark, or copied layouts. This matters because you said you want to eventually share this publicly; an original teaching tool that stands on its own is worth building, an imitation isn’t.

4. **Zero real-account risk, by design.** Nothing in this course touches your actual GitHub account or real repos unless and until you explicitly choose to graduate a lesson onto real github.com yourself, later, outside the app.
5. **Every tier ends with a real skill, not a quiz.** Same rhythm as the CLI dojo — you did the thing, a grader confirms you did the thing, you get a badge.
6. **Continuous with the dojo, not a reboot.** Same vocabulary (CURRENT TRUTH, sealed record, disposition), same “sandbox that can’t hurt anything” promise, same owner-in-the-loop framing. One path, one product, not a new thing wearing the old name.
7. **Never gate the visual course behind the command line.** The six visual tiers are the default front door — a brand-new learner should be able to start on Tier 1 with zero terminal experience and never feel blocked. Commands are hard on day one for almost everyone; forcing that friction first is how courses lose people before they ever see the payoff. The command line is offered, not required.

3. How this maps to what you already have

The CLI dojo you finished isn’t a prerequisite anymore — it’s a permanent, optional feature living inside the same app: the **Command Test Center**.

	Command Test Center (was: CLI Git Dojo)	The Six Visual Tiers (this document, Tiers 1-6)
Where it runs	An in-app terminal, built into the same web app	The same web app, visual/interactive screens
Teaches	The commands, the mechanics, the loop	The interface, the context, the “why here”
Format	The same 7 lessons + graders you already completed, ported in as-is	Visual simulated GitHub screens + graders
When you touch it	Anytime, by choice — a persistent button reachable from every screen	The default path from the moment you open the app
Required to proceed?	No. Skip it entirely and still finish every visual tier.	This is the course.
Status	Content already built and tested (v1.0.2) — this is a porting job, not a rewrite	Not yet built — this is the plan

One app, one badge shelf, two independent tracks. A learner can finish all six visual tiers having never opened the Command Test Center, or can bounce into it after Tier 1 out of curiosity, or can ignore the visuals entirely and grind commands first if that’s genuinely how they learn best. Nothing about the app should assume there’s only one right order.

4. What's actually on GitHub's screen right now (research basis)

Before designing lessons around screens, I confirmed what GitHub's real interface currently looks like, since it changed meaningfully in the last year and I won't teach a stale picture:

- **Global navigation** was redesigned around a hamburger-style expandable menu (quick links to Issues, Pull Requests, Discussions, your teams, and most-used repos), a persistent breadcrumb trail showing exactly where you are in the site hierarchy, and a unified search/command palette that jumps to code, repos, orgs, or people from one box. Issues, PRs, and notifications stay one click away in the upper right on every page.
- **File browsing and editing** got smoother: an "edit on default branch" option appears directly from search results instead of a dead disabled button, and a new "view on default branch" control sits next to the branch picker so switching branches while browsing files is obvious instead of confusing.
- **The pull request "Files changed" page** — the single most important screen for anyone reviewing work — now has a resizable file tree on the left (with indicators for which files have comments, errors, or warnings) beside the diff on the right, a dedicated Overview panel for the PR description, and a review panel that shows your pending comments before you submit so you can double-check your own feedback. You can now comment on any line, not just lines next to a change.

This course is built against *that* interface, not a screenshot from three years ago. (Sources at the bottom of this document.)

5. The tiers

Tier 1 — See the Interface

Goal: a total beginner can open GitHub.com and know what they're looking at.

Module	What it teaches	Why it matters to you specifically
1.1 What GitHub actually is	Git = the tool that tracks changes (what the dojo taught). GitHub = a website that hosts a copy of that tracked history and adds a social/visual layer on top. Same relationship as Lesson 6's <code>hub/website.git</code> — just real instead of fake.	Removes the single most common beginner confusion: "isn't Git and GitHub the same thing?" No.
1.2 The repository home screen	File tree, rendered README, the green "Code" button, About panel, topics, branch picker, commit-count link — one labeled	This is the screen a future contractor, or you six months from now, sees first. It should never feel

	screen, every element explained.	foreign.
1.3 Reading history visually	The commit graph — same information as git log, drawn instead of typed. Clicking a commit to see exactly what changed.	Pairs with Lesson 2 of the Command Test Center (“The Ledger”) if you’ve tried it — same custody-trail concept, new view. Stands on its own if you haven’t.
1.4 Repo settings basics	Public vs. private, description, default branch, danger zone (why it’s called that).	The first real decision point: which of your repos should ever be public.
1.5 The global nav	Hamburger menu, breadcrumbs, notification bell, unified search — current 2026 layout.	So you’re never lost, and can find any repo or notification in one motion.

Tier 2 — Collaborate Like a Team

Goal: understand every screen involved when someone else’s work needs your review.

Module	What it teaches
2.1 Issues	Anatomy of an issue — title, body, labels, assignee, milestone, linked PR. The “this is broken / this needs doing” record.
2.2 Opening a pull request	What a PR actually is (a <i>proposal</i> to change main — the same idea as a branch, taught fresh here, with a screen and a comment thread wrapped around it).
2.3 The Files Changed page	The current redesigned diff view — resizable file tree, inline comments on any line, the review panel, draft comments saved locally so you never lose feedback.
2.4 Reviewing & ruling	Approve / Request changes / Comment — the visual version of the owner’s-seat judgment call from Lesson 7’s capstone.
2.5 Merge strategies	Merge commit vs. squash vs. rebase — what each does to history, and why “never rewrite shared history” (a rule you already learned) points you toward specific choices.

Tier 3 — Protect the Trunk

Goal: understand the governance controls that make “main is CURRENT TRUTH” a mechanically enforced fact, not a polite request.

Module	What it teaches
	Require a PR before merge, require approvals, require

3.1 Branch protection rules	passing checks — literally the machine enforcing owner approval, screen by screen.
3.2 CODEOWNERS	A file that says “changes to this path always need this person’s review” — automatic routing, mapped directly to a real RTS.AI scenario (e.g., only you approve changes to anything under /governance).
3.3 Roles & permissions	Read / Triage / Write / Maintain / Admin — who can do what, and why a future contractor gets Write, not Admin.
3.4 Secrets & security basics	Never commit a credential (Lesson 7’s planted API key, revisited on the real security-alerts screen), .gitignore, what a secret-scanning alert looks like.
3.5 Releases & tags	Marking an actual shipped version with a changelog — the difference between “committed” and “released.”

Tier 4 — Run Automation

Goal: understand what a GitHub Action is well enough to read one, trust one, and know when to ask for one.

Module	What it teaches
4.1 What a workflow is	The “robot that runs your checks automatically every time something changes” mental model, before any YAML.
4.2 Anatomy of a workflow file	Triggers, jobs, steps — read, not yet written from scratch.
4.3 A first real check	A workflow that runs a simple automated check on every PR; watching the Actions log turn from yellow (running) to green (passed) or red (failed) directly on the PR screen.
4.4 Environments & secrets for automation	Where credentials for automated jobs live, and why they’re never typed into a workflow file directly.
4.5 Where this could plug into RTS.AI’s world	Honest, non-overclaiming discussion of what automated checks <i>could</i> eventually support (e.g., a file-naming or format check on every PR) — framed as a future option, not a current product feature.

Tier 5 — Run the Organization

Goal: the “holding my company files” tier — everything about GitHub built for more than one person.

Module	What it teaches
	Why a company’s repos belong

5.1 Personal account vs. Organization	in an Organization, not your personal account, and what changes when you make that move.
5.2 Teams & permissions at scale	Grouping people once instead of granting access repo-by-repo, forever.
5.3 Repository templates & standards	Keeping every new repo consistent (README shape, .gitignore, LICENSE) without re-deciding it each time.
5.4 Projects (the board)	The kanban/table/roadmap view that ties Issues and PRs together into one picture of “what’s actually happening across everything.”
5.5 Pages, Wikis, Discussions	Where documentation and public-facing pages live — direct tie to your website repo.
5.6 Full lifecycle case study	Lesson 7’s capstone — the contractor delivery with a planted bug and a planted secret — replayed entirely on these visual screens instead of the terminal. Same review, same ruling, same disposition record, now in the seat you’ll actually sit in.

Tier 6 — Publish What You Learned

Goal: the “teach myself, then share with the world” close-out.

Module	What it teaches
6.1 Preparing a repo for public eyes	README quality, LICENSE choice, CONTRIBUTING.md, topics, a social preview image — the difference between a repo that works and one that welcomes a stranger.
6.2 Publishing the dojo itself	Walking your own completed git-dojo folder through everything just learned — real account, real repo, your first genuine public commit.
6.3 Completion ledger	A final screen showing every tier, every badge, every score — your own certificate, generated from what you actually did, not a participation trophy.

6. Progression, badges, and scoring

Carried over from what you asked for: **tiered learning badges up to expert**, with **the app actually logging completion and scoring accordingly** — not a vague “nice job” screen. Revised in v1.1 to run as **two independent tracks on one shelf** instead of one forced sequence:

- **Track A — The Six Visual Tiers.** Tiers 1 through 6, each ending

in a hands-on task graded automatically. A badge unlocks per tier, only when every module in that tier passes. This is the main path, and it's fully completable without ever touching Track B.

- **Track B — The Command Test Center.** The same 7 lessons you already completed, ported into the app as a standalone practice arena with its own 7 badges (one per lesson) plus a bonus badge for finishing all seven. Reachable at any time via a persistent control visible from every screen in the app — not buried in a menu, not something you have to leave the visual course to go find.
 - **No cross-blocking, either direction.** Track A never requires Track B to be started, attempted, or finished. Track B never requires any Track A progress either — someone could open the app and go straight to commands if that's genuinely how they learn best.
 - **Soft cross-links, not gates.** Where a visual lesson has a natural command-line counterpart (e.g., Tier 1.3's commit graph ↔ Command Test Center Lesson 2), the lesson can *offer* a "try this in the Command Test Center" link — an invitation, never a requirement, never a locked door.
 - **A visible progress ledger** at all times, showing both tracks side by side — which tier you're in, what's done, what's next, and the Command Test Center's status — because you specifically don't like "some hidden score I can't see."
 - **v1 scope honesty:** for a single user (you), progress can persist locally in the app without needing real user accounts. If this ever goes public with multiple learners, that's a v2 decision (real accounts, a real database) — flagged now so the build doesn't paint itself into a corner, not something to solve today.
-

7. Visual identity

Recommendation, not yet decided by you: give this its own identity in the same visual family as your website (dark charcoal base, one accent color, clean industrial-modern feel) but **not** badged as an official RTS.AI governed product — this is a teaching tool, separate from the governed B2B Desk, same posture as the website build. Working name for now: **"Git Dojo"** (matches what you already called it) with the visual system extended from the Command Test Center's terminal mockups into full simulated-GitHub screens for Tiers 1-6.

8. What happens next

The companion document —

PROMPT_GitHub_Mastery_Path_Replit_Build_v1_1_2026_08_14.md — turns every section above into a complete, self-contained build order for a fresh Replit Agent chat, the same way the website prompt worked. It also carries the porting instructions for turning your completed git-dojo_v1_0_2.zip into the in-app Command Test Center, so attach that zip alongside the prompt when you start the Replit chat. Nothing here commits you to anything; read it, change what doesn't fit, then hand it to Replit when you're ready.

Sources

- [Exploring GitHub with the redesigned navigation, now in public beta — The GitHub Blog](#)
- [Improved file navigation and editing in the web UI — GitHub Changelog](#)
- [Improved pull request "Files changed" page on by default —](#)

[GitHub Changelog](#)

- [GitHub for Beginners: Getting started with GitHub Actions — The GitHub Blog](#)
 - [About protected branches — GitHub Docs](#)
 - [Managing a branch protection rule — GitHub Docs](#)
-

RTS.AI Owner Training Series · GitHub Mastery Path Curriculum v1.1
· draft for owner review, not yet built